Formal Methods Meet 2

“Program testing can be used to show the presence of bugs, but never to show their absence”- Dijkstra

Types of Formal Verification

Static:

- Design time is offline
- Covers all cases(exhaustive) but at the same time costly
- Important in security domain as even one edge case might lead to threats

Dynamic/Runtime:

- Deployed after the product is in the market
- Only limited cases are being tackled
- But does the work fast

Difference between Verification and Monitoring

- Its checking vs logging
- It is formally checking something specific
- Monitoring is adding everything without any conditions
- There are many disagreements in the community regarding the different between these 2 terms

  Runtime Verification aka Trace Analysis, Dynamic Analysis

Runtime Verification composition:

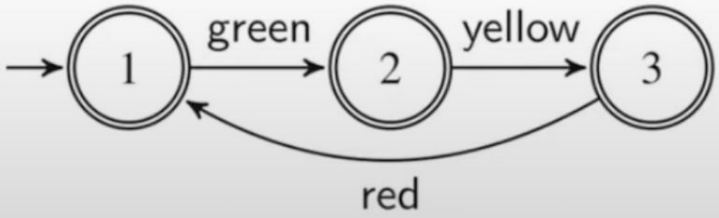
- Event -> transitions/actions in the system

  State changes, timeouts, triggers like light turning red, pedestrian button is pressed

- Traces -> what the system does/performs/behaves

  Set of events like Red->Yellow->Green

- Formal Specification -> specifying the conditions and states (FSM, Automaton, Regex)
- Monitors -> Observe the system and verify if required properties are satisfied or not



$$(\text{green yellow red})^*(\text{green} \mid \text{green yellow} \mid \varepsilon)$$

$$\Box(\text{green} \rightarrow \circ\text{yellow} \wedge \text{yellow} \rightarrow \circ\text{red} \wedge \text{red} \rightarrow \circ\text{green})$$

Types of monitoring

Eg Here an action can only be done after login

1) **Complete synchronous**: A login(from any task) unblocks all tasks, an action without login gives violation to every task (rules applied to all tasks and not specific)
2) **Synchronous monitoring w/ sync instrumentation**: Specific for every task, gives running/violation to every task differently according to requirement.
3) **Async monitoring w/ checkpoints:** Get all requests asynchronously, when critical checks are being done checkpoints are sent to thread and are kept blocked, and then the system resolves all tasks at hand.
4) **Async Monitoring w/ synchronous detection:** Get all requests asynchronously, all threads run independently, and if detection of violation, synchronously block all threads and do the needed(like violation) and then resume/unblock rest of threads.
5) **Completely asynchronous monitoring :** a buffer is kept, receiving the events and handles them independently.

How to verify?

- Local Quantification (LQ) : checkpoints
- Temporal Quantification (TQ): relative points
- Data Quantif (DQ): values across states as a parameter with TQ
- Abstract Data Specification (ADS): abstract states and variables

Approaches:

- Assertions(LQ,TQ,DQ,ADS) statements at user specified locations in code
- Pre/Post Conditions:(TQ,DQ,ADS) usually placed before or after methods
- Invariants(LQ): properties to be held at all locations
- State Machine and Process Algebras (ADS)
- Temporal Assertions (TQ) - related to time and data quantification
- Domain Specific Languages(ADS): complete formal specification language

Introduction to Runtime Verification

Ref - Lectures on Runtime Verification by Ezio Bartocci, Yliès Falcone

Runtime verification(also referred as runtime monitoring) lightweight, yet rigorous, formal method that complements classical exhaustive verification techniques with a more practical approach that analyses a single execution trace of a system.

Steps to check whether the runtime behaviour of a system conforms to some specification:

- Specifying (Un)Desired System Behaviour
- Generating Monitor from a Specification
- Connecting a Monitor to a System

The Abstract Notion of Behaviour

Any kind of observation about the system is termed as an **event**. Eg- lightTurnsGreen

- This refers to the way the system changes over time, by updating some internal state, taking some internal or external action, or affecting the environment in some way.
- We can either inspect some snapshots of the current state of the system at a particular time, or record certain actions or state changes made by the system.
- Events are discrete atomic entities. There can be two alternative views -

1. It has a duration(start and end time): *eg- we might have the event lightGreen at one level of abstraction, but lightGreenOn and lightGreenOff at another level of abstraction.*
2. In languages specified over continuous time events can be viewed as signals that can be queried for a value.

- **Alphabet** the set of the system's observable events of interest.
- Events, which are chosen during system specification, can be defined at different levels of abstraction to represent a system's internal or external behavior, from a whole system down to a specific part, and don't have to directly match existing system components or actions.

Traces

- We use events to abstract a particular observation about a system. We abstract the behaviour of a single run of a system as a **trace**, which is a (finite) sequence of events.
- Traces can be finite, representing an observed run, or infinite, representing possible future behaviors.
- A trace can consist of a single event at each point in time or a set of events, which is useful for describing concurrent observations where the exact order is not important.
- Traces can also be structured to include quantitative time, not just the order of events.

Properties and Specifications

- A property is an abstract concept that defines a set of all acceptable traces for a system.
- A specification is the concrete, textual description of a property.
- While multiple specifications can describe the same property, a property is unique and independent of its specification language.
- The goal of a specification is not to describe the system's entire behavior, but rather the key behaviors at a useful level of abstraction.

Explicit Properties: These are properties explicitly written in a specification language.

Implicit Properties: These are properties not formally specified but are monitored using custom algorithms like deadlock avoidance.

General Specification Language Features

- Executable vs Declarative:

  Executable languages (e.g., state machines) are straightforward to monitor but are often low-level and less abstract where as Declarative languages (e.g., temporal logic) are more abstract, easier to combine, but require a separate step to generate a monitor.

- Prefix Closure:

  This feature is common for safety properties, where any partial execution that is a prefix of a valid trace is also considered valid. Most languages don't assume this, but some do.

- Language-to-property relationship:

  Sometimes, for usability reasons, it might be useful for a language to not directly describe the property being specified, but be implicitly related to it. Therefore, this is less a feature of a language and more a feature of its usage. (see examples next slide)

Polarity: Specifications can describe either good behavior (validation) or bad behavior (violation).

Suffix Matching: A specification might only describe a pattern that, if found at the end of a trace, indicates a violation.

- Finite vs Infinite:

  Some languages are better suited for specifying finite traces (e.g., state machines), while others are better for infinite traces (e.g., temporal logic). Since runtime observations are always finite, a mapping from infinite to finite semantics is often needed.

- Time:

  Qualitative Time: The order of events in a trace gives a qualitative sense of time, but can be fragile if not all events are recorded.

  Quantitative Time: Languages can incorporate quantitative time (e.g., "30 seconds") using explicit clocks or intervals to specify concrete time durations between events.

- Data and Quantification:

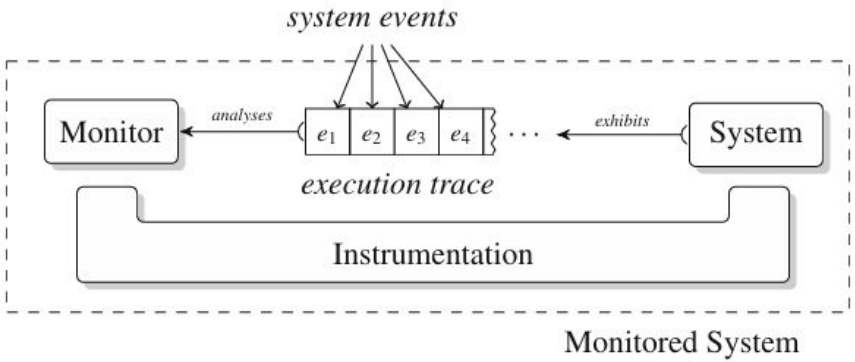
  Languages can treat events as atomic symbols or as structures with data. They can include features for data manipulation and quantification over data, such as guards and assignments in state machines, or various forms of quantifiers in temporal logics.

The Monitoring Setup

It mainly consists of three components-

- the system-under-scrutiny
- the monitor
- the instrumentation mechanism

The collective is referred to as the monitoring setup.



Monitors

- A monitor is a computational entity that runs alongside a system to observe its behavior.
- It determines if a specific property is being satisfied or violated.
- It can issue a definite verdict (e.g., "accepted" or "rejected") once enough information is gathered. This verdict is usually permanent and cannot be revised.
- Monitors are considered part of the trusted computing base (TCB). They must be correct, reliable, and interfere minimally with the system being monitored.
- Monitors are typically created through an automated synthesis process from a formal specification of the property. This process helps ensure their correctness and standardizes their code.

Instrumentation

Instrumentation is the "computational plumbing" that links the system's execution to the monitor's analysis.

Aspect 1: Event Recording

- It identifies and captures relevant parts of the system's execution (e.g., variable changes, function calls, memory operations).
- It transforms these actions into a stream of events, which form the execution trace fed to the monitor.
- Instrumentation can either filter and redirect existing observable behavior or extract previously unobservable data, turning a "black-box" system into a "grey-box" one.

> The recorded events are then reported to the monitor in the form of an ordered stream called an **execution trace**.

Aspect 2: Execution Synchronization

- It manages how the system and the monitor run in relation to each other.
- This can range from synchronous execution (regulated by a clock) to asynchronous execution with defined synchronization points. This choice impacts the timeliness of a monitor's detections.

Monitor-Oriented Programming (MOP)

MOP is a code design principle that separates the core system functionality from monitoring and other "ancillary" concerns like safety and security.

It's a layered architecture with the core system at the center and monitors in outer layers.

In MOP, monitors are more powerful. They can not only analyze traces and report violations but also actively influence the system's behavior by:

- Suppressing or filtering system output.
- Injecting "adaptation actions" to change the system's future behavior or structure.

Monitoring Modes

OFFLINE	ONLINE
• Process: Analysis is performed after the system finishes executing. Events are stored in a file or database. • Advantages: Less intrusive, lower overhead is not a concern. Allows for global trace analysis, including backward traversal, since the entire trace is available. • Disadvantage: Late detections—violations are only discovered after the system run is complete, preventing real-time mitigation of issues.	• Process: Analysis is performed during system execution. • Advantages: Enables early detection and potential real-time response to violations. • Disadvantages: Operates under strict constraints, like low overheads and the need to analyze partial traces incrementally.

Online Monitoring Strategies

SYNCHRONOUS	ASYNCHRONOUS
• Process: The system and monitor execute in lock-step. After generating an event, the system pauses and waits for the monitor to process it before continuing. • Advantage: Guarantees timely detection and strong guarantees about the relationship between system and monitor state. • Disadvantage: Highly intrusive, leading to significant overhead on the system's execution.	• Process: The system and monitor run independently, on separate threads. • Advantage: Less intrusive, resulting in lower overheads for the system. • Disadvantage: Can still have late detections if the monitor falls behind the system, especially on platforms without fair execution guarantees.

Many systems use a mix of these strategies to balance overhead and detection timeliness.

Monitor Architecture

MONOLITHIC	DECENTRALIZED
• Concept: A single, large monitor is synthesized to represent the entire system specification. • Benefit: Simpler to synthesize and manage.	• Concept: Multiple monitors work together to check different parts of a specification or different data sources. • Types: Component-based: Monitors are localized to specific event sources (e.g., individual components of a system). Concurrent: Monitors run in parallel on multi-core hardware to improve performance.

Distributed Monitor Coordination

ORCHESTRATION	CHOREOGRAPHY
• Concept: A single, central entity collects, orders, and analyzes events from various distributed sources. • Advantage: Simpler to synthesize. • Disadvantage: Can lead to high network traffic and a single point of failure.	• Concept: Monitoring tasks are distributed across multiple monitors that coordinate with each other. • Advantage: Reduces network traffic and increases fault tolerance. • Disadvantage: More complex to design and implement.

Monitor Code Placement

INCLINING (Incline Monitoring)	SEPERATION (Outlined Monitoring)
• Process: Monitor code is directly inserted into the system's application code. • Advantages: Lower overheads and greater expressiveness as it has direct access to the system's internal state.	• Process: Monitor code is a separate, distinct unit that receives events from the system. • Advantages: Minimally alters the original code, is less error-prone, can be offloaded to other machines, and simplifies compositional analysis.

Instrumentation

Mechanism to probe and extract info/signals, traces from software or hardware system DURING EXECUTION.

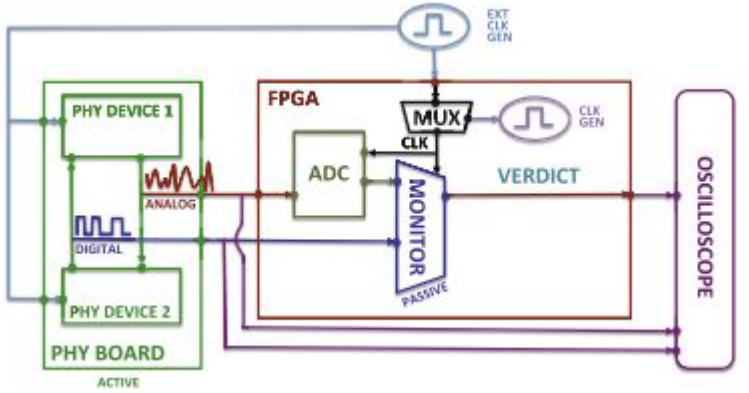
2 types of instrumentation:

- **Hardware**: requires extracting analog signals using physical wires
- **Software**: implemented on low level language in compiled form (bytecode/assembly)

Hardware instrumentation

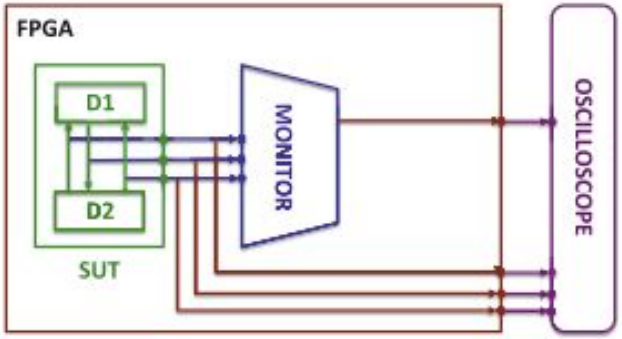
Due to the increasing complexity of new generation of hardware, simulation of such hardware for testing purposes has become hard and requires FPGA platforms to complement them.

External Monitoring



- External clock is shared between the emulation and the monitor
- Both analog(later converted to digital) and digital signals are processed synchronously and outputted to the oscilloscope

Internal Monitoring



Here, both the emulation and monitor are implemented in the same hardware.

JTAG

- Popular standard test architecture used for on-chip instrumentation and debugging.
- It uses a serial interface to sample signals from a chip's pins and can also be used to drive output signals.
- JTAG gives a debugger control over the processor, allowing it to be halted, single-stepped, or run freely.(by using embedded trace buffers)

Software Instrumentation

- Consists of adding an extra code to track the execution and monitor components.
- Can be performed at either source code or binary code level
- Can be static(compilation time) or dynamic(tracking dynamically linked libraries)
- Nowadays AOP(aspect oriented programming) codes(advices) contained in file(aspect) and processed by aspect weaver.
- Cons:
  - If some code is not reached, SI cant cover any info for that.
  - Computational overhead
  - Heisenbugs(bugs that disappear or alter in the SI)

- Solutions: limited no of events selected, statistical models used to work in probability rather being discrete.

Monitorability

- If a property is monitorable, worth of monitoring it during the run time.
- Examples where monitorability is required or not:

  If a property p always holds-> here anytime p doesn't hold, the monitor can observe and be sure it is not followed.

  If a property p holds then q property holds in the future-> here when p becomes true, then there can be infinite states where q is not true but becomes true after n transitions, here monitor cannot execute properly since it doesn't know exactly if the property q will hold or not in the future.

Definitions of Monitorability

Monitoring to detect bad behaviours(Kim et al.):

Something bad should never happen. Any bad execution whether its infinite will have a finite prefix where the monitor becomes sure that the sequence is bad/invalid and no future good behaviour can change that.

Formally: a property $\phi \subseteq \Sigma\omega$ is said to be monitorable if $\Sigma*$ \pref(ϕ) is co-recursively enumerable(can be identified by algos), where pref(ϕ) is the set of prefixes of ϕ.

Ex: yellow->red always okay but as soon as green->red is found, return error.

Definitions of Monitorability

Monitoring to detect good and bad behaviours(Pnueli and Zaks):

Monitor should be able to confirm both good and bad behaviour ($\top/\bot$).

Positively Determined: A trace σ positively determines a property if every possible infinite continuation of σ satisfies the property. At this point, the monitor can confidently give a True verdict.

Negatively Determined: A trace σ negatively determines a property if every possible infinite continuation of σ violates the property. The monitor can then give a False verdict.

Formally: ϕ is said to be σ-monitorable, if σ has a continuation such that ϕ is either positively or negatively determined by this continuation. A property is monitorable if it is σ-monitorable, for any $\sigma \in \Sigma*$

Ex: open(file) will be unknown since we dont know what will happen, close(file) will be make the property true but in future we can get another open(file) so its undetermined.

Definitions of Monitorability

Monitoring with a semantics for finite executions(Falcone et al.):

Taking consideration that there can be situations where monitor reaches a final verdict and thus we need to remove (?) state. This is possible for property to be satisfied for finite test case but not infinite.

Formally: A property is monitorable if the way it's evaluated on correct vs. incorrect finite traces is different enough that a monitor can reliably tell when to issue a final, permanent verdict.

Eg: Open(file) is CURRENTLY state $\perp$ and close(file) is state CURRENTLY $\top$(don't know about future, ultimate final verdict unknown).

Characterizations of Monitorable Properties

To determine if a property is monitorable by identifying the class to which it belongs, often just by analyzing its syntactic structure.

Based on Kim et al.

This approach defines monitorable properties as the class of safety properties.

- A key requirement is that the set of invalid prefixes must be co-recursively enumerable (i.e., there is an algorithm to identify them).

Based on Pnueli & Zaks(Most Studied)

- This is a broader characterization that forms a strict superset of the union of safety and co-safety properties.
- It is also a strict superset of obligation properties, which are combinations of safety and co-safety properties.
- The set of properties monitorable under this definition is closed under countable unions and intersections.
- Adding more types of verdicts does not expand the class of monitorable properties.

Based on Falcone et al. (Finite Trace Semantics)

- This characterization is more restrictive, forming a **strict subset** of obligation properties.
- However, its power is revealed through its verdict system. When using a 4-valued truth-domain ($\{\bot, \bot c, \top c, \top\}$), this definition allows any linear-time property to be monitored